PLAYWRIGHT PARA LEIGOS

Colinha do curso · os comandos e a receita de um teste, pra consultar enquanto pratica

COLINHA · 1/5

1 TERMINAL: OS COMANDOS

`node -v`	confere se o Node está instalado
`npm init playwright@latest`	cria o projeto e instala o Playwright
`npx playwright test`	roda todos os testes (sem janela)
`npx playwright test --ui`	abre o modo UI: você vê cada passo
`npx playwright show-report`	abre o relatório HTML
`npx playwright codegen site`	grava seus cliques e escreve o código
`npx playwright install`	baixa os navegadores (Linux: `--with-deps`)

2 ONDE FICA CADA COISA

```
meu-primeiro-playwright/
├─ tests/            ← os seus testes
│  ├─ login.spec.js
│  └─ compra.spec.js
├─ pages/            ← Page Objects (aulas 37 a 40)
├─ node_modules/     ← não mexa
├─ playwright.config.js ← as configurações
├─ .env              ← usuário e senha
├─ package.json      ← a ficha do projeto
└─ .gitignore
```

Todo arquivo de teste termina em `.spec.js`, é assim que o Playwright sabe que é teste.

3 A RECEITA DE UM TESTE

```
import { test, expect } from '@playwright/test';

test.describe('Login', () => {          // o caderno de receitas
  test.beforeEach(async ({ page }) => {  // roda antes de CADA teste
    await page.goto('/');
  });

  test('entra com usuário e senha', async ({ page }) => {
    await page.getByTestId('username').fill('standard_user');
    await page.getByTestId('password').fill('secret_sauce');
    await page.getByTestId('login-button').click();
    await expect(page.getByTestId('title')).toHaveText('Products');
  });
});
```

Todo teste é assim:

1. **abre** o site (goto)
2. **acha** o elemento (getBy...)
3. **faz** alguma coisa (fill, click)
4. **confere** o resultado (expect)

await = "espera esse passo terminar". Esqueceu o `await`? O teste fica estranho e falha sem motivo.

4 AÇÕES

`page.goto(url)`	abre o site
`.fill('texto')`	digita num campo (apaga o que tinha)
`.click()`	clica
`.selectOption('valor')`	escolhe opção de um menu
`.check()`	marca uma caixinha
`.press('Enter')`	aperta uma tecla
`.first()`	pega só o primeiro da lista

5 LEIA O EXPECT COMO UMA FRASE

`expect(...)`	**eu espero que**...
`toHaveURL(/inventory/)`	...tenha no endereço
`toBeVisible()`	...esteja visível
`toContainText('txt')`	...contenha o texto
`toHaveText('txt')`	...tenha exatamente o texto
`toHaveCount(2)`	...tenha essa quantidade (count = contar)
`not.toBeVisible()`	...**não** esteja visível

Sempre com `await` na frente. O Playwright espera sozinho (por padrão, até 5 segundos) a condição ficar verdadeira.

Site de treino: saucedemo.com · usuário **standard_user** · senha **secret_sauce**

PLAYWRIGHT PARA LEIGOS

Seletores, ferramentas que ajudam e como rodar só o que importa

COLINHA · 2/5

6 COMO ACHAR O ELEMENTO (AULAS 17 E 18)

1º MELHOR getByTestId

`getByTestId('username')`

feito pra testes: não muda quando o visual muda. No saucedemo, configure `testIdAttribute: 'data-test'`

1º TAMBÉM getByRole

`getByRole('button', { name: 'Login' })`

acha como uma pessoa acharia: pelo papel e pelo nome

2º texto ou rótulo

`getByText('Products')`

`getByLabel('Email')`

bom pra textos que não mudam

EVITE `locator('div:nth-child(3) > button')`: depende da posição na página. Mudou o layout, o teste quebra à toa.

7 FERRAMENTAS QUE AJUDAM

Codegen	você clica no site e ele escreve o código
Pick locator	aponta pro elemento e ele te dá o seletor
Modo UI	roda e mostra cada passo, com foto
Trace Viewer	a "caixa-preta": o filme do teste que falhou
Modo debug	`--debug` ou `await page.pause()`: para no meio
Relatório	lista tudo que passou e falhou, com print e vídeo

8 RODANDO SÓ O QUE IMPORTA (AULA 25)

`test.only(...)`	roda só esse teste
`test.skip(...)`	pula esse teste
`npx playwright test login`	só os arquivos com "login" no nome
`-g "bloqueado"`	só os testes com esse texto no título
`--project=chromium`	só nesse navegador
`--headed`	mostra o navegador abrindo

Não esqueça o `test.only` no código: no GitHub Actions ele faz tudo falhar de propósito.

9 O PLAYWRIGHT.CONFIG.JS QUE A GENTE MONTA (AULAS 23 E 24)

```
import dotenv from 'dotenv';  // lê o arquivo .env (usuário e senha fora do código)
dotenv.config({ path: path.resolve(__dirname, '.env') });

export default defineConfig({
  use: {
    baseURL: 'https://www.saucedemo.com',   // aí o goto('/') já sabe o site
    testIdAttribute: 'data-test',            // o getByTestId procura data-test
    trace: 'on-first-retry',  screenshot: 'only-on-failure',  video: 'retain-on-failure',
  },
});
```


Site de treino: saucedemo.com · usuário **standard_user** · senha **secret_sauce**

PLAYWRIGHT PARA LEIGOS

Erros comuns, organização profissional, Git e as palavras do dia a dia

COLINHA · 3/5

10 DEU VERMELHO? LEIA O ERRO ASSIM (AULA 26)

1
Qual linha?
o erro mostra o arquivo e a linha, com uma setinha

2
O que ele esperava?
"Expected" é o que você pediu, "Received" é o que tinha

3
O que tinha na tela?
abra o trace ou o print no relatório

`Timeout 5000ms exceeded`	não achou o elemento: confira o seletor e o data-test
`Expected ... Received ...`	achou, mas o texto é outro: confira a mensagem do site
`Executable doesn't exist`	faltam os navegadores: `npx playwright install`
`npx não é reconhecido`	reabra o VS Code; se continuar, reinstale o Node (aula 7)
`scripts desabilitados`	`Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`

11 ORGANIZAÇÃO PROFISSIONAL

Page Object	cada página vira uma classe com seus campos e ações
test.step	divide o teste em etapas com nome, no relatório
storageState	faz login uma vez e reaproveita nos testes
.env	usuário e senha fora do código (nunca suba pro GitHub)
for ... of	o mesmo teste pra vários usuários

12 GIT E GITHUB (AULAS 43 A 45)

`git init`	liga o Git na pasta (só 1 vez)
`git add .`	separa todos os arquivos
`git status`	mostra o que mudou
`git commit -m "msg"`	salva o jogo, com um nome
`git push`	manda pro GitHub

O `npm init playwright` já cria o `.github/workflows/playwright.yml`: subiu pro GitHub, os testes rodam sozinhos. Senha vai em **Settings › Secrets**.

13 GLOSSÁRIO: AS PALAVRAS DO DIA A DIA

QA: quem cuida da qualidade do sistema, testando antes de chegar no cliente

Spec: um arquivo de teste (o seu `login.spec.js`)

Asserção: a conferência do teste, o `expect`

Projeto (project): um navegador ou aparelho configurado no config

Trace: a gravação passo a passo de um teste

API: a porta dos fundos do sistema, que conversa por dados

E2E: de ponta a ponta, usa o sistema como um usuário de verdade

Locator: o "endereço" de um elemento na página

Headless: navegador rodando sem janela, mais rápido

Flaky: teste instável, às vezes passa, às vezes falha

CI: testes rodando sozinhos sempre que o código muda

Regressão: rodar tudo de novo pra ver se nada quebrou

Continua na página 4 ›

PLAYWRIGHT PARA LEIGOS

Page Object, login salvo, GitHub Actions e testes de API

COLINHA · 4/5

14 PAGE OBJECT: CADA PÁGINA VIRA UMA CLASSE (AULAS 37 A 40)

```
// pages/LoginPage.js
export class LoginPage {
  constructor(page) {
    this.page = page;
    this.usuario = page.getByTestId('username');
    this.senha = page.getByTestId('password');
    this.botaoLogin = page.getByTestId('login-button');
  }
  async entrar(usuario, senha) {
    await this.usuario.fill(usuario);
    await this.senha.fill(senha);
    await this.botaoLogin.click();
  }
}
```

```
// no teste
import { LoginPage }
  from '../pages/LoginPage';

const login = new LoginPage(page);
await login.entrar(
  'standard_user', 'secret_sauce');
```

Mudou o site? Você conserta **num lugar só** (a classe), e todos os testes que usam ela continuam funcionando.

15 LOGIN UMA VEZ SÓ (AULA 42)

```
// tests/auth.setup.js: faz login e guarda
await page.context().storageState({
  path: 'playwright/.auth/usuario.json',
});

// nos testes: já começa logado
test.use({ storageState:
  'playwright/.auth/usuario.json' });
```

No config, o projeto `setup` roda antes (`dependencies: ['setup']`). A pasta `playwright/.auth` fica no `.gitignore`.

16 TESTES DE API (AULAS 46 A 48)

`async ({ request })`	pede o request no lugar do page
`request.get(url)`	buscar
`request.post(url, { data })`	criar
`request.delete(url)`	apagar
`resposta.status()`	o número da resposta (com parênteses!)
`await resposta.json()`	os dados que vieram

200 deu certo · **201** criado · **404** não encontrado. Sem navegador: roda em milissegundos.

17 GITHUB ACTIONS: OS TESTES RODANDO SOZINHOS (AULA 45)

```
# .github/workflows/playwright.yml
    - name: Run Playwright tests
      run: npx playwright test
      env:
        SAUCE_USER: ${{ secrets.SAUCE_USER }}
        SAUCE_PASSWORD: ${{ secrets.SAUCE_PASSWORD }}
```

1. **Settings › Secrets and variables › Actions**
2. **New repository secret**: `SAUCE_USER` e `SAUCE_PASSWORD`
3. `git push` e abre a aba **Actions**
4. Verde: passou. O relatório fica pra baixar no fim.

Secret = segredo: ninguém vê o valor depois.

PLAYWRIGHT PARA LEIGOS

Nível 2: telas diferentes e o Playwright de verdade no trabalho

COLINHA · 5/5

18 TELAS DIFERENTES (AULAS 56 A 63)

`.check()` / `.uncheck()`	marque / desmarque
`.selectOption('2')`	escolha a opção
`.setInputFiles('arquivos/x.txt')`	anexe o arquivo
`page.on('dialog', d => d.accept())`	alerta: **antes** do clique
`page.waitForEvent('popup')`	aba nova: **antes** do clique
`page.frameLocator('#id')`	dentro do iframe
`.hover()`	passe o mouse
`.dragTo(destino)`	arraste até
`.filter({ hasText: 'Ana' })`	a linha com este texto

E sempre confere: `toBeChecked()`, `toHaveValue('2')`, `toBeVisible()`.

19 FIXTURES: O TESTE RECEBE PRONTO (AULA 64)

```
// fixtures/paginas.js
export const test = base.extend({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  },
});
// no teste: pede igual ao page
test('...', async ({ page, loginPage }) => {
```

★ A LOJINHA DO LEIGO (AULAS 66 A 80)

tsvini.github.io/custo-oculto-fabrica/loja
Usuários: **cliente** e **bloqueado** · senha **leigo123**
`?versao=1` no fim do endereço: a versão com o bug

20 DADOS NUM ARQUIVO (AULA 65)

```
import usuarios from '../../dados/usuarios.json';
for (const { usuario, resultado } of usuarios) {
  test(`${usuario} - ${resultado}`, ...);
}
```

Caso novo = **uma linha no JSON**. Passos diferentes? Testes separados.

21 MOCK: O PLAYWRIGHT NO MEIO (AULA 66)

```
// servidor fora do ar
page.route(api, (r) => r.abort());
// responde no lugar do servidor
page.route(api, (r) => r.fulfill({
  status: 200, body: '[]' }));
```

Testa como a **tela** reage. Se o servidor funciona: teste de API, sem mock.

22 VISUAL, SOFT E TAGS (AULAS 67 A 69)

`toHaveScreenshot('login.png')`	compare com a foto guardada
`--update-snapshots`	mudou de propósito: atualize a foto
`expect.soft(...)`	anote o erro e continue conferindo
`{ tag: '@smoke' }`	a etiqueta (depois do nome do teste)
`--grep @smoke`	rode só os marcados
`--grep-invert @smoke`	rode todos, menos os marcados

23 FLAKY, AMBIENTES E EVIDÊNCIA (AULAS 70 A 72)

`waitForTimeout(3000)`	× tempo fixo: lento e ainda falha
`toBeVisible({ timeout: 10000 })`	✓ espera **até** aparecer
`--repeat-each=10`	roda 10 vezes: caça teste instável
`process.env.BASE_URL || '...'`	o ambiente vem de uma variável

Card de bug: título claro · passos · esperado × obtido · **print, vídeo e trace** anexados. Retry diminui o estrago, não corrige a causa.

Teste visual no GitHub: no config, `ignoreSnapshots: !!process.env.CI`